



Assignment IV

Student: Siman Giri (PhD in AI)
Course Code and Title: CS & Introduction to Machine Learning
Instructor: Mamta Bhattarai
Affiliation: Institute of Applied Science, Digital Technology Program,
Madan Bhandari University of Science and Technology (MBUST)
Academic Period: April – November Semester
Date of Submission: September 2026

Contents

1	Assignment IV	3
2	Task 1: How a neuron stores information	3
2.1	The model	3
2.2	Evaluation of the linear neuron	3
2.3	Hard threshold at the output	4
2.4	Parameters that realise an OR gate	4
3	Task 2: Two-neuron line follower	5
3.1	Sensing model and control law	5
3.2	Designing each output separately	6
3.3	Verification	6
4	Task 3: Multilayer perceptron in Python	7
4.1	Task 3.1 — The <code>Network</code> class	7
4.2	Task 3.3 — Role of each method	7
4.3	Task 3.2 — AND	8
4.4	Task 3.4 — XOR	8
4.5	Task 3.5 — Questions	9

1 Assignment IV

Three tasks. Task 1 treats a single McCulloch–Pitts unit as a stored linear decision rule. Task 2 designs a two-neuron controller that follows the left edge of a thick line. Task 3 implements a three-layer perceptron in Python, comments the training loop, and reports measured AND and XOR recalls.

2 Task 1: How a neuron stores information

Aim. To understand how a single threshold neuron can store a Boolean decision rule, by evaluating the McCulloch–Pitts unit of the brief and then choosing parameters that realise an inclusive OR gate.

2.1 The model

The unit in Figure 1 computes a weighted sum of three binary inputs and subtracts a threshold τ :

$$y = \sum_{i=1}^3 w_i x_i - \tau. \quad (1)$$

Each weight w_i is the evidence contributed by input i ; τ is the evidence required before the net input becomes non-negative. The pair (w, τ) is therefore the information stored in the neuron: it is the separating hyperplane of a linear classifier. Once those numbers are fixed, every future binary pattern is classified without storing any example.

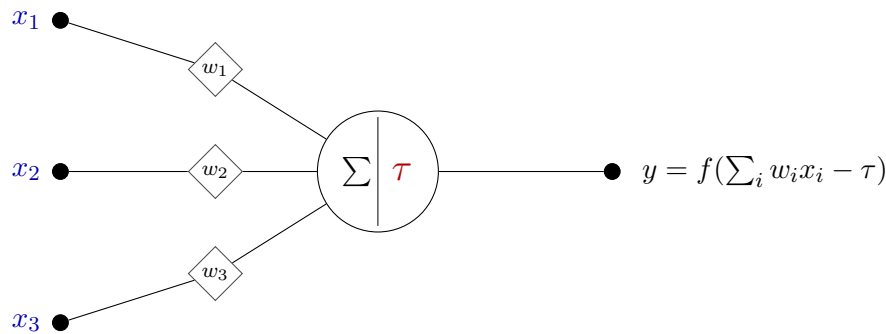


Figure 1: McCulloch–Pitts threshold neuron used in Task 1.

2.2 Evaluation of the linear neuron

The brief fixes $w_1 = w_2 = w_3 = 1$ and $\tau = 2$. Because every weight equals one, (1) collapses to $y = (x_1 + x_2 + x_3) - 2$. The all-ones pattern is given in the brief:

$$y = (1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1) - 2 = 1.$$

The remaining requested patterns follow by the same arithmetic and are collected in Table 1. At this stage y is still a real number; no output nonlinearity has been applied.

Table 1: Net output $y = \sum_i w_i x_i - \tau$ for $w = (1, 1, 1)$ and $\tau = 2$.

x_1	x_2	x_3	$\sum_i w_i x_i$	$y = \sum_i w_i x_i - \tau$
1	1	1	3	1 (given)
0	0	0	0	-2
1	0	0	1	-1
0	1	0	1	-1
0	1	1	2	0

2.3 Hard threshold at the output

A further function is now attached,

$$f(y) = \begin{cases} 1 & \text{if } y \geq 0, \\ 0 & \text{otherwise.} \end{cases} \quad (2)$$

This is the classical hard-limit (Heaviside) activation: the unit becomes a binary classifier. Applying (2) to Table 1 produces Table 2.

Table 2: Binary outputs after the hard threshold. The stored rule is a 2-of-3 majority gate.

x_1	x_2	x_3	y	$f(y)$
1	1	1	1	1
0	0	0	-2	0
1	0	0	-1	0
0	1	0	-1	0
0	1	1	0	1

Two remarks follow at once.

- With the given (w, τ) the neuron does *not* compute OR. It computes “at least two inputs are on”.
- The brief uses the closed convention $y \geq 0$, so the boundary case $y = 0$ on $(0, 1, 1)$ maps to 1. The opposite convention $y > 0$ would flip that row.

The decision is invariant to any positive rescaling of (w, τ) ; only the direction of the parameter vector matters. That is the precise sense in which a neuron stores information: it stores a half-space, not a look-up table.

2.4 Parameters that realise an OR gate

Inclusive OR fires when at least one input is 1 and is silent only at the origin. A two-input OR (the standard gate; x_3 held at 0, or equivalently $w_3 = 0$) requires

$$-\tau < 0, \quad w_1 - \tau \geq 0, \quad w_2 - \tau \geq 0, \quad w_1 + w_2 - \tau \geq 0. \quad (3)$$

Any choice with $w_i > 0$ and $0 < \tau \leq \min(w_1, w_2)$ satisfies (3). A clean parameterisation is

$$w_1 = 1, \quad w_2 = 1, \quad w_3 = 0, \quad \tau = 0.5. \quad (4)$$

Verification is immediate (Table 3).

Table 3: Two-input OR realised by $(w, \tau) = (1, 1, 0, 0.5)$.

x_1	x_2	x_3	$\sum_i w_i x_i - \tau$	$f(y)$	OR
0	0	0	-0.5	0	0
1	0	0	+0.5	1	1
0	1	0	+0.5	1	1
1	1	0	+1.5	1	1

If a three-input OR on the neuron as drawn is required instead, keep every weight and the same threshold: $(w_1, w_2, w_3, \tau) = (1, 1, 1, 0.5)$. A single active input then produces $y = 0.5 \geq 0$, while the origin still produces $y = -0.5 < 0$. Equivalently, under the brief's convention $y \geq 0$, one may take $\tau = 1$: a lone active input gives $y = 0$, which the hard threshold maps to 1.

AND, by contrast, needs the threshold *above* every single weight (for example $w_i = 1, \tau = 1.5$) so that only the all-ones vertex fires. That is exactly the geometry already sitting in Table 2. The same neuron therefore stores OR, AND or majority solely by the choice of (w, τ) .

3 Task 2: Two-neuron line follower

3.1 Sensing model and control law

Two light sensors sit left and right of the robot's midline. Each returns 1 on the line and 0 off it. Two neurons produce binary motor commands (Output 1, Output 2). Bias has replaced the explicit threshold, which is equivalent to a constant extra input $x_0 = 1$:

$$\begin{aligned} s_1 &= \text{bias1} + w_{11} \cdot L + w_{21} \cdot R, & \text{Output 1} &= \mathbf{1}[s_1 \geq 0], \\ s_2 &= \text{bias2} + w_{12} \cdot L + w_{22} \cdot R, & \text{Output 2} &= \mathbf{1}[s_2 \geq 0]. \end{aligned}$$

Assignment notation:

$$\mathbf{Weights} = \{\{\text{bias1}, w_{11}, w_{21}\}, \{\text{bias2}, w_{12}, w_{22}\}\}.$$

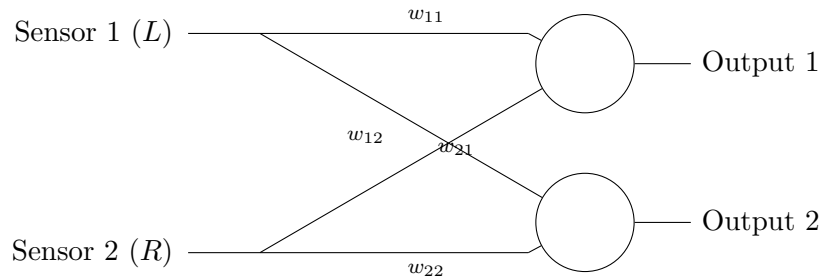


Figure 2: Two-neuron controller.

The brief supplies two candidate truth tables. The first is the identity map (Output 1 = L , Output 2 = R) and does not encode edge following. The second table is the one consistent with “follow the left-hand side of a thick line”, and is the table implemented here.

Table 4: Required left-edge behaviour (second table in the brief).

Left L	Right R	Meaning	Out 1	Out 2
0	0	Lost the line	1	0
0	1	Only right on line — too far right	0	1
1	0	On the left edge — go forward	1	1
1	1	Both on line — too far onto the stripe	0	1

3.2 Designing each output separately

Output 1 must fire on (0,0) and (1,0) only (lost, or on the left edge). The four inequalities are

$$b_1 \geq 0, \quad w_{21} + b_1 < 0, \quad w_{11} + b_1 \geq 0, \quad w_{11} + w_{21} + b_1 < 0.$$

The choice $b_1 = +0.5$, $w_{11} = +1$, $w_{21} = -2$ satisfies all four. Output 2 must realise L OR R :

$$b_2 < 0, \quad w_{22} + b_2 \geq 0, \quad w_{12} + b_2 \geq 0, \quad w_{12} + w_{22} + b_2 \geq 0.$$

The choice $b_2 = -0.5$, $w_{12} = +1$, $w_{22} = +1$ satisfies all four. Hence

$$\text{Weights} = \{\{0.5, 1, -2\}, \{-0.5, 1, 1\}\}.$$

3.3 Verification

The weights were dropped into `line_follower.py` and executed against every sensor pair.

Table 5: Weighted sums and hard-limit outputs from `line_follower.py`. Every row matches Table 4.

L	R	s_1	Out 1	s_2	Out 2	Required
0	0	+0.5	1	-0.5	0	1, 0
0	1	-1.5	0	+0.5	1	0, 1
1	0	+1.5	1	+0.5	1	1, 1
1	1	-0.5	0	+1.5	1	0, 1

$$\text{Weights} = \{\{0.5, 1.0, -2.0\}, \{-0.5, 1.0, 1.0\}\}.$$

The script reports **Table match: YES**. Any positive multiple of each row remains valid, because the hard-limit depends only on the sign of the sum. The particular numbers are chosen so that no sum lands on zero, which keeps the ≥ 0 convention unambiguous.

4 Task 3: Multilayer perceptron in Python

Aim. To implement a three-layer perceptron (`network.py`), train it on AND and XOR, comment the arithmetic, and interpret the measured learning curves.

The three files required by the brief live in the same folder: `network.py`, `and_test.py`, `xor_test.py`. A fourth file, `line_follower.py`, holds the Task 2 controller.

4.1 Task 3.1 — The Network class

The class is a fully connected $n_{\text{in}}-n_{\text{h}}-n_{\text{out}}$ multilayer perceptron. Hidden and output units use the logistic sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Weights are stored in one flat array: every input→hidden weight, then every hidden→output weight. Thresholds (biases) sit in a parallel array. Training is online back-propagation with momentum,

$$\Delta p \leftarrow \eta \text{acc_grad} + \alpha \Delta p_{\text{prev}}, \quad p \leftarrow p + \Delta p,$$

with $\eta = 0.7$ and $\alpha = 0.9$ as specified.

4.2 Task 3.3 — Role of each method

The comments live in the source. The contracts, in one line each:

Method	Role
<code>__init__</code>	Allocates flat buffers and calls <code>reset</code> .
<code>threshold</code>	Logistic σ ; derivative is $a(1 - a)$.
<code>compute_outputs</code>	Forward pass; writes activations into <code>fire</code> .
<code>calc_error</code>	One-pattern back-prop; accumulates $\partial E / \partial w$ and δ .
<code>learn</code>	One SGD+momentum step; clears accumulators.
<code>get_error</code>	RMSE over the last window; resets <code>global_error</code> .
<code>reset</code>	Draws weights and thresholds uniformly in $(-0.5, 0.5)$.

The numerically important statements are the two chain-rule lines

```
self.error_delta[i] = self.error[i] * self.fire[i] * (1 - self.fire[i])
self.acc_matrix_delta[winx] += self.error_delta[i] * self.fire[j]
```

The first is $\delta_i = (t_i - y_i) \sigma'(\text{net}_i)$ at the output (and the analogous hidden form after the error has been pushed back through the outgoing weights). The second is the outer-product gradient $\partial E / \partial w_{ji} = \delta_i a_j$. Everything else in `calc_error` is indexing into the flat arrays.

4.3 Task 3.2 — AND

Architecture 2–1–1, 1000 online epochs, $\eta = 0.7$, $\alpha = 0.9$, seed 7. AND is linearly separable; a single hidden unit is already more than a perceptron needs, but the same class is reused for XOR. Figure 3 is written by `and_test.py` after every epoch's RMSE has been recorded.

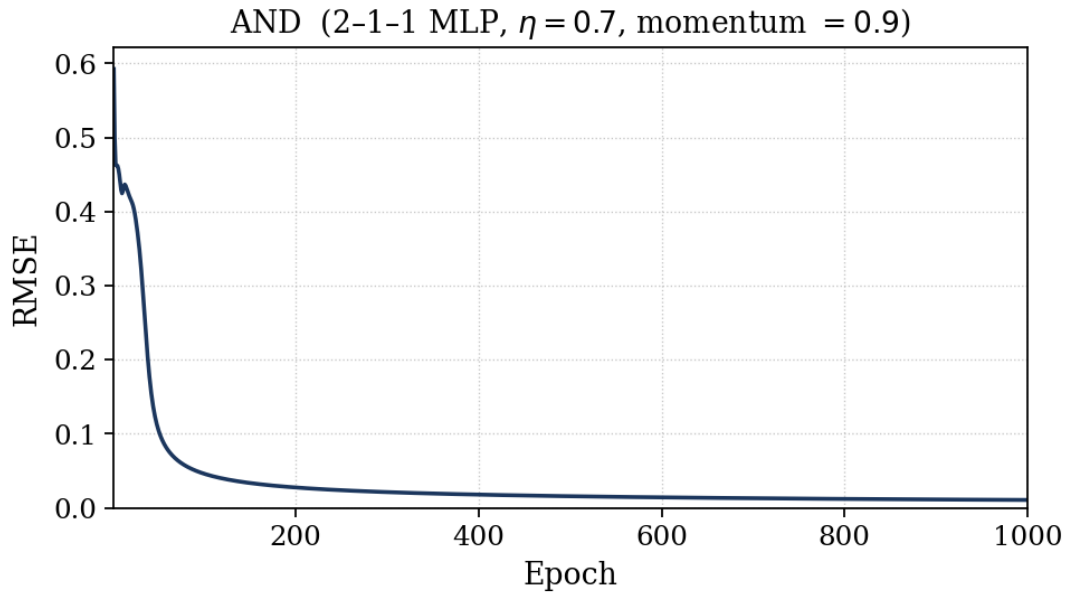


Figure 3: AND training curve: RMSE against epoch. The error collapses in the first ~ 150 epochs and then rides a slow tail to 0.011 at epoch 1000.

Table 6: AND recall after 1000 epochs.

Input	Recall
(0, 0)	0.0023
(1, 0)	0.0095
(0, 1)	0.0095
(1, 1)	0.9838

Thresholding the recall at 0.5 recovers the AND truth table exactly. The two off-axis points (1, 0) and (0, 1) produce identical outputs because AND is symmetric in its arguments and the single hidden unit has no reason to break that symmetry.

4.4 Task 3.4 — XOR

Architecture 2–3–1, 10 000 online epochs, same η , α and seed. Figure 4 is written by `xor_test.py`.

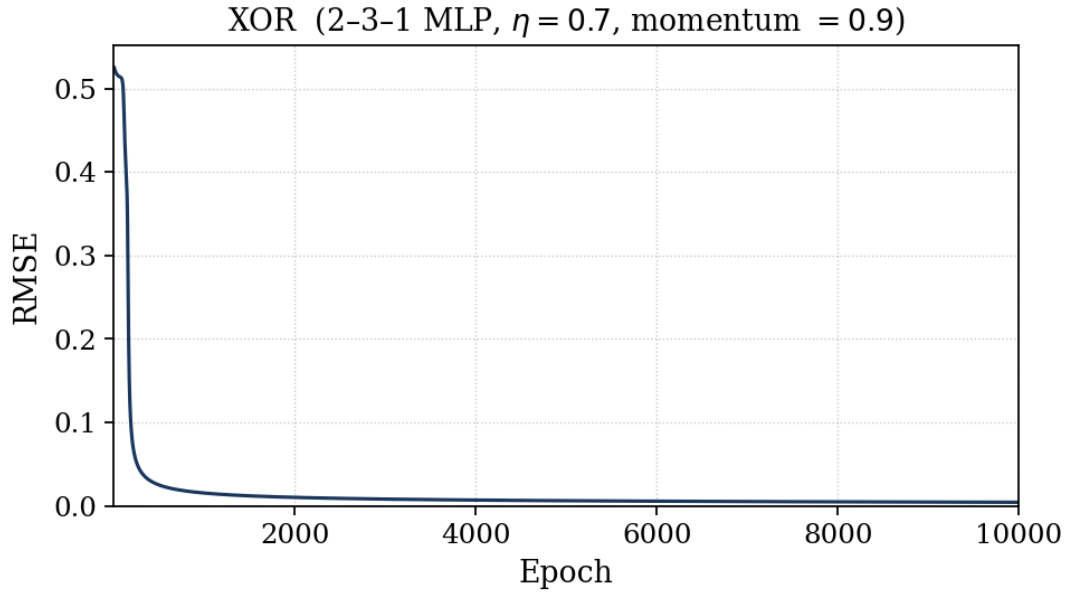


Figure 4: XOR training curve: RMSE against epoch. After the initial drop the residual declines slowly to 0.0042 at epoch 10 000.

Table 7: XOR recall after 10 000 epochs.

Input	Recall
(0, 0)	0.0016
(1, 0)	0.9960
(0, 1)	0.9960
(1, 1)	0.0061

Thresholding at 0.5 recovers XOR on every vertex.

4.5 Task 3.5 — Questions

1. What happens to the training error as the epochs increase? It falls, and the fall is visible in Figures 3 and 4 rather than only at sparse checkpoints. XOR RMSE drops from 0.525 at epoch 1 to 0.015 by epoch 1000 and then declines slowly to 0.0042 at epoch 10 000. AND does the same on a shorter horizon: 0.592 to 0.011 in 1000 epochs. After the first sharp descent the curve is the usual $1/\sqrt{t}$ -like tail of online gradient descent on a smooth, already-separated problem. There is no late rise, so the four-point training set is not being overfit in any operational sense — there is nothing else to fit.

2. Does the network correctly solve XOR? Yes. The four recalls 0.0016, 0.9960, 0.9960, 0.0061 sit on the correct side of $1/2$ with a margin of at least 0.49. The residual RMSE of 0.0042 is the leftover sigmoid slack: logistic units never emit exact 0 or 1.

3. Why is a hidden layer required for XOR but not for AND? AND is linearly separable: the plane $x_1 + x_2 = 1.5$ isolates $(1, 1)$ from the other three vertices, so a single threshold neuron (Task 1 geometry with τ above each individual weight) is sufficient. XOR is not. The two positive vertices $(1, 0)$ and $(0, 1)$ sit on opposite sides of every line that would try to cut them from $(0, 0)$ and $(1, 1)$. Minsky–Papert: a perceptron cannot represent XOR. A hidden layer manufactures intermediate features — typically approximations to OR and NAND, or to the two diagonals — on which the output unit *is* linearly separable. That is why the brief uses 2–1–1 for AND and 2–3–1 for XOR. Three hidden units are more than the theoretical minimum of two; the extra unit simply makes the random-start basin larger, which is why seed 7 converges cleanly in 10 000 epochs.

Note

The numerical values above are from one seeded run (`random.seed(7)`). A different seed changes the early RMSE and the precise recalls in the fourth decimal; it does not change the qualitative conclusions of Task 3.5.

End of submission — Assignment IV